Антифрод по метаданным вызовов

Архитектура и использованные технологии · Хакатон Nexign «Телеком будущего», кейс «Антифрод» · Команда [название команды]
Репозиторий: **https://github.com/Bibon123/antifraud-cdr**

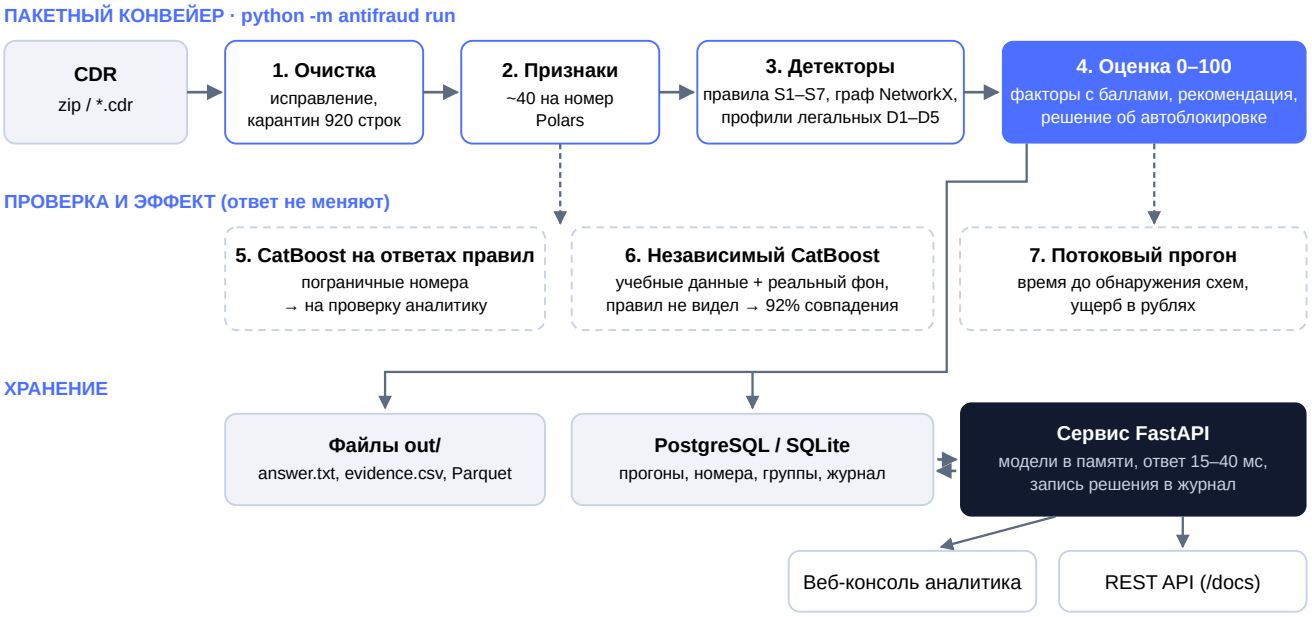
1. Что делает решение

Сервис анализирует записи о вызовах (CDR) — кто, кому, когда, сколько длился вызов и куда переадресован — и находит номера семи схем мошенничества из кейса (Wangiri, робот-обзвон, SIM-box, клон SIM, цепочки переадресаций, карусель, вишинг со спуфингом). Пять типов легальных массовых источников (колл-центры, банковские оповещения, ночные диспетчеры, горячие линии, оповещения ЖКХ/МЧС) распознаются отдельно и не попадают в ответ. Каждое решение раскладывается на факторы с баллами; автоблокировка — отдельное решение с отдельными условиями; вызовы на 112 и 101–104 соединяются всегда.

621	0 из 359	92%	17 мин	15–40 мс
мошеннический номер, все 7 схем	легальных номеров задето	ответа подтвердила независимая модель	до обнаружения Wangiri в потоке	ответ API по одному вызову

Данные кейса: 471 файл, 2 353 333 записи за 1–5 октября 2026 года, 374 689 номеров.

2. Архитектура



Пакетный конвейер запускается одной командой и выполняет семь шагов. Шаги 1–4 формируют ответ; шаги 5–7 проверяют его и считают эффект, но ответ не меняют. Результаты пишутся в файлы и в базу данных. Сервис читает номера из базы, держит обе модели в памяти и отвечает на запрос «оцени вызов» за 15–40 мс.

3. Компоненты

Модуль	Назначение
`ingest.py`	чтение архива, номера как строки (ведущие нули Альфы), исправление ошибок формата, карантин неисправимых строк с причиной
`features.py`	поведенческий профиль номера: объём, длительности, собеседники, последовательность номеров B, время суток, наложение вызовов, IMSI, переадресации
`detectors.py`	правила схем S1–S7; графы звонков и переадресаций (карусели, цепочки, связи вишинга, кампании роботов); профили легальных источников D1–D5
`scoring.py`	оценка 0–100 как сумма вкладов факторов, рекомендации, автоблокировка (оценка ≥ 95, ≥ 2 независимых фактора, нет невиновного владельца), экстренные номера
`ml.py`, `ml_synth.py`	две модели CatBoost с объяснением через SHAP: на ответах правил (пограничные номера) и независимая — на размеченных учебных данных с реальным фоном
`synth.py`, `evaluate.py`	генератор учебных CDR с известной разметкой и проверка устойчивости правил при сдвиге параметров
`realtime.py`, `economics.py`	воспроизведение записей в порядке времени, момент обнаружения каждой схемы, ущерб и предотвращённый ущерб в рублях
`db.py`	база данных (SQLAlchemy): прогоны, номера, группы, журнал решений; PostgreSQL или SQLite с той же схемой
`app/main.py`, `index.html`	сервис FastAPI и веб-консоль аналитика: обзор, проверка звонка, номера, группы, качество, журнал
`config.toml`	все пороги, тарифы, экстренные номера и белый список — меняются без правки кода

4. Технологии

Технология	Зачем
Python 3.13	язык реализации; стек задан кейсом
Polars	столбцовая обработка 2,35 млн записей: признаки всех номеров за секунды
NetworkX	графовые схемы, которые не видны по одному номеру: карусели, цепочки переадресаций, связи вишинга
CatBoost	независимая проверка ответа и подсказки аналитику; встроенный SHAP объясняет вклад признаков
scikit-learn	кросс-валидация; запасная модель, если CatBoost недоступен
FastAPI, Uvicorn	REST API и веб-консоль; интерактивная документация API на `/docs`
SQLAlchemy, PostgreSQL	база результатов и журнал решений; без сервера — SQLite с той же схемой
pytest	19 автотестов: очистка, все схемы, ловушки, скорая, API, база, экономика
Docker Compose	описан запуск PostgreSQL и сервиса одной командой (на нашей стороне не собирался)

5. Путь одного вызова через API

`POST /api/score_call {a, b, ts, dur}` → номера приводятся к 10 цифрам → если B — экстренная служба, вызов соединяется всегда → оценка звонящего берётся из базы → обе модели считаются по признакам номера из хранилища признаков → политика выбирает действие (блокировать / остановить и проверить / предупредить / соединить) → решение с факторами пишется в журнал. Неизвестный номер получает «нет данных», а не «риск 0». При недоступности сервиса вызовы проходят как обычно (fail-open).

6. Запуск

- Windows: двойной щелчок по `start.bat` — создаёт окружение, ставит библиотеки, считает и открывает консоль.
- Вручную: `pip install -r requirements.txt`, затем `python run_all.py путь\к\cdr_out.zip`; консоль — `http://127.0.0.1:8000`.
- Полный прогон — около 6 минут на 2 ядрах, пик памяти 3,2 ГБ. Python 3.11–3.14, рекомендуется 3.13.

7. Качество и ограничения

Разметки в данных нет, поэтому качество проверено четырьмя способами: независимая модель (574 из 621, 92%), согласованность на реальных данных (0 пересечений с 359 легальными номерами), устойчивость на учебных данных со сдвигом параметров (100% в пяти сценариях, полнота 82%, если мошенники втрое тише) и автотесты. Ограничения: пороги правил подобраны по описанию кейса; подмена номера определяется косвенно, поэтому подменённые номера маркируются, а не блокируются; потоковый режим — эмуляция; денежная оценка зависит от тарифов оператора.